State fold: the n = 83 K600 root index stores each unordered pair once

2026-10-15

State fold: the n = 83 K600 root index stores each unordered pair once

Status (2026-10-15): measured; frozen primary rule applied; verdict `improvement`. GitHub Actions run 38819494806 (attempt 1, one `ubuntu-24.04` runner, Tier 1 instance scope) ran the frozen protocol and schedule. Its artifact `n83-state-fold-38819494806-1` is archived in `run_github_38819494806/`. On 16 new points the folded index lowers the mean cold cost (setup + one-target online) by **1.8372 [1.8216, 1.8528]** (cold cB / cF). See Result.

Question

The n = 83 a = 1 K600 one-target IC (crypto `770f6a69` `koblitz_orbit_dlp_fast_online.rs` with B's `ic_log_cache.patch`) spends about 30 s of per-process setup, nearly all of it building the root index of 29,880,000 states (600 × 600 × 83) and its 4 GiB table. The scan-order note (`../koblitz-n83-scanorder-20261015/README.md`, "Next improvements") observed that (A, B) and (B, A) are duplicates. Does storing each unordered class once lower the **cold** cost (setup + one-target online) on new targets, with the same relation, scalar and table? This is an iteration under AGENTS.md "Success criterion"; the rank stage is measured separately.

Symmetry analysis (exact, from the source)

`build_index128` pushes state (l, r, rel) for every left orbit l, right orbit r and relative shift rel, with roots x(A ± τ^rel B), A = reps[l], B = reps[r] (the two roots of S3(x(A), x(τ^rel B), X)). Every state is regular at K600 (`regular_states` = 29,880,000 in every receipt).

- **Pair swap.** The mate (r, l, (n − rel) mod n) has roots x(B ± τ^−rel A) = Frob^−rel x(A ± τ^rel B). The S3 solver is symmetric in its two inputs and Frobenius-equivariant (squaring, the half trace and the trace test all commute with Frobenius, and b = 1 is fixed), so the mate's normal-basis roots are the representative's rotated by the mate's rel, in the same root order. Over its n shifts the mate makes exactly the representative's (absolute, partner) probes, with the first two x-codes swapped, so it hits iff the representative hits, with the same relation up to the order of its first two points. (l, l, 0) is its own mate.
- **No further state-level symmetry.** Sign (±B) is already inside a state (both roots), and the diagonal Frobenius is already quotiented by fixing A at shift 0; a shift of B alone changes rel, which is a different class. The signed-Frobenius relative shift therefore gives no extra fold.
- **Table.** The root table already holds each root orbit once: the mate is always inserted after its representative (larger left, or the same left with rel ≥ 42) and its roots are rotations, so its `insert_if_absent` calls were no-ops. Folding leaves the key → value map unchanged. The table has 29,878,182 entries, not 29,880,000: 1,818 root orbits are shared by two different classes (a four-term relation among base points); that is the same in both builds.
- **Count.** l < r: C(600, 2) · 83 = 14,915,100; l = r with 2 rel ≤ 83: 600 · 42 = 25,200. **14,940,300 stored states, from 29,880,000** (0.50002). The table capacity follows the stored count as the baseline's follows its own, so it falls from 2^27 to 2^26 slots (4.0 to 2.0 GiB).

The same fold already exists for the narrow (n ≤ 64) path in crypto `770f6a69` (`build_index(..,pair_symmetry=true)`, its `pair_symmetry_tests`), which keeps every scan position. Here the wide path stores only the representatives.

The fix

`make_online_source.py` derives the producer from the combined producer (`claude/n83-combined` @ 88e1929fb, lift_scan × Feistel; `combined_derivation.py` is that branch's derivation verbatim and must reproduce its frozen digest `f65e518a...`) by counted exact replacements in `mod wide`:

- `KIC_STATE_FOLD=1` builds the 14,940,300 representatives (one S3 solve and two canonicalizations each) and fills the table from them in the baseline's insertion order.
- **The online scan is the baseline's, exactly.** It walks the 29,880,000 logical state indices in the same order (left-major from `start`, or the same keyed Feistel bijection on the logical count) and rebuilds a mate by two rotations. A state whose mate came earlier in that order is skipped (the Feistel position of the mate comes from the new `invert`). The mate made the same probes without a hit, so the first relation found is the baseline's: same x-codes, intermediates, point indices and scalar; only the probe counter drops by the skipped states' probes.
- `KIC_FOLD_PLANT=1` (planted positive control) redoes and discards every mate's S3 solve and canonicalizations in the build, and rescans every skipped state online into `fold_audit_probes`, which must make up the probe difference exactly.
- `KIC_INDEX_CANON=lift` (secondary) canonicalizes the index build with lift_scan (the same values; the next improvement to try).
- `KIC_RANK_CANON`, `KIC_RANK_ORDER` select the rank stage's canonicalizer and order and record per-attempt probes and ns; `KIC_INDEX_DIGEST=1` writes, after the target and outside every interval, a blake3 digest of all logical states in index order and an order-independent digest of the table map. Every IC process must reproduce B's digests.

Frozen design

One GitHub `ubuntu-24.04` runner, one pinned CPU, Tier 1 counters, one fresh process per run. Cold panel: 16 new public points (role `statefold-cold-v1`, `make_targets.py`), checked disjoint from Q0, panel B, the 384 scan-order held-out points and the 576 combined primary points.

Arm	Index	Canonicalizer / order	Role
`cB`	unfolded	lift / Feistel	baseline (combined producer)
`cF`	folded	lift / Feistel	candidate
`cFaa`	folded	lift / Feistel	A/A
`cX`	folded + planted work	lift / Feistel	planted positive control
`cBL`, `cFL`	unfolded, folded	lift / left-major	lift_scan single arm
`cBF`, `cFF`	unfolded, folded	full / Feistel	Feistel single arm
`cFI`	folded, lift in build	lift / Feistel	next improvement
`rT`	—	—	tuned rho rT770, seed 850000 + point seed
`kB`, `kN`	unfolded / folded	rank: full + left-major / lift + Feistel	rank stage, two blocks, then one target

- **Primary:** R = mean cold(cB) / mean cold(cF), cold = setup + online of one target in one process, bootstrap over the 16 points (10,000 draws). Improvement iff the lower bound > 1; regression iff the upper bound < 1. Invalid unless every gate passes, A/A cF/cFaa is inside [0.97, 1.03] and the planted cX/cF lower bound exceeds 1.
- **Gates:** B's state count, stored count, table entries and digests in every IC process; one scalar per point across every arm and rT, replayed in the pinned oracle; one relation per order group; probes cB = cBF, cF = cFaa = cX = cFF = cFI $\leq$ cB, cX + audit = cB, cFL $\leq$ cBL; rank arms at full rank with a factor-log table equal to B's archived table.
- **Secondary (never the verdict):** setup, online, index and peak-RSS ratios; the single-arm pairs and the better unfolded single arm over cF (the baseline if run 38803625711 does not promote the combination); cF/cFI; the rank stage (kB/kN totals, per-attempt cost ratio); cold `S` with and without the rank stage, old and new, and the cold and online gaps to rT.

- **Power** (`power_check.py`, `analysis/power.json`, prior from run 38786433493): 8 points already give power 1.0 at a 5% cold effect; 16 are run for the secondaries.
- **Runtime estimate:** about 1.6–3 h including the build; 325 min time guard.

Local validation (cloud-agent VM; no timing claims)

- The derivation reproduces the combined producer's digest, then builds (rustc 1.97.0 here; the runner pins 1.99.0) with no new warnings. All 9 producer tests pass, including the new ones: Feistel `invert` on whole domains and the real S; the fold map's counts and bijectivity; and, on a 9-orbit n = 83 base, every rebuilt state equal to the unfolded state, identical table maps and digests, and on 24 planted four-point targets per order the same relation, `probes(fold)+audit=probes(unfolded)`, with skips exercised.
- Full K600 on Q0 (one process each): `cB`, `cF`, `cX` give the same relation [77273, 22971, 54962, 62539] and scalar 3141592653589793 as the archived Feistel run; probes 1,988,184 (cB, = archived) and 1,987,852 (cF, one skipped state), cX audit 332 (sum 1,988,184); identical state and table digests; peak RSS 5,543 → 2,812 MiB.
- `smoke/`: the runner, wrapper, gates and verifier on segment C1.

Result

All gates pass: every scheduled row is present and valid, 16/16 points complete, probe identities, one relation per order group, one scalar per point across every arm and the tuned rho (replayed in the pinned oracle), rank tables equal to B's archived table. The A/A and the planted control pass. `verify_receipt.py` re-run on the archive reproduces `result.json` byte for byte. The verifier flags one contention note: serial 3 (`cF`) recorded major page faults; the row is valid and stays in.

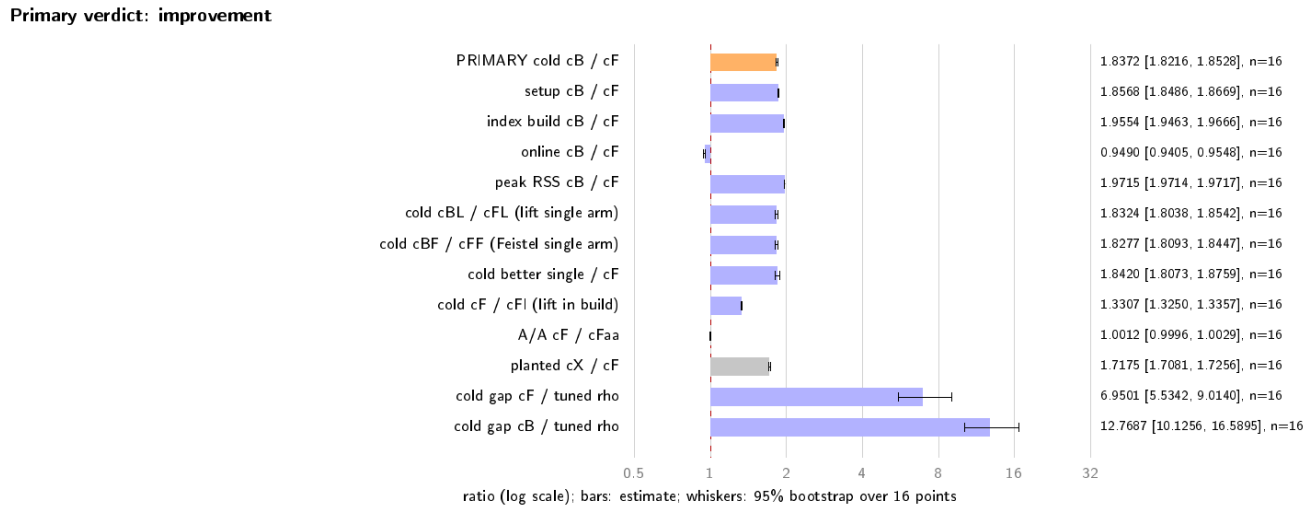


Figure 1: Frozen ratios with 95% bootstrap intervals, all from `result.json` (source, vector, rendered by `make_figures.py`).

Primary (frozen rule)

Improvement iff the 95% lower bound of mean cold(cB) / mean cold(cF) exceeds 1.

Statistic (n = 16 new points)	Estimate [95%]	Reading
mean cold cB / mean cold cF	1.8372 [1.8216, 1.8528]	**improvement**
A/A cF / cFaa	1.0012 [0.9996, 1.0029]	pass (inside [0.97, 1.03])
planted cX / cF	1.7175 [1.7081, 1.7256]	pass (lower bound above 1)

Mean cold times (ms): cB 41,953.4; cF 22,835.6; cFaa 22,808.8; cX 39,220.3.

Secondaries (none changes the verdict)

Quantity	cB	cF	Ratio cB / cF [95%]
setup (ms)	41,485.8	22,342.9	1.8568 [1.8486, 1.8669]
index build (ms)	39,183.0	20,038.5	1.9554 [1.9463, 1.9666]
online (ms)	467.6	492.7	0.9490 [0.9405, 0.9548] (geomeans 0.9539 [0.9455, 0.9630])
peak RSS (MiB)	5,544.3	2,812.1	1.9715 [1.9714, 1.9717]

- **Online is not faster; it is about 5% slower** with the fold (the mate is rebuilt by two rotations on the scan). The gain is in setup and memory. The cold gain is nearly all index build, as the analysis predicted.
- **Single-arm pairs.** Cold cBL / cFL (lift_scan, left-major) 1.8324 [1.8038, 1.8542]; cold cBF / cFF (full scan, Feistel) 1.8277 [1.8093, 1.8447]. Better unfolded single arm (cBL) over cF: 1.8420 [1.8073, 1.8759]. So the fold's effect does not depend on which parent is the base.
- **Next improvement.** Canonicalizing the index build with lift_scan (cFI): cF / cFI = 1.3307 [1.3250, 1.3357]; cFI mean cold 17,160.5 ms (secondary; not a frozen arm of the primary).
- **Cold S** (setup + online, tuned-rho steps over $\sqrt{r}$, this run's measured rT770 step of 279.8 ns): cB 1.6196, cF 0.8816, cFI 0.6625. Delta S cold cB to cF = −0.7380.
- **Gap to tuned rho** (IC / rho, 16 points). Cold, without the rank stage: cB 12.7687 [10.1256, 16.5895], cF 6.9501 [5.5342, 9.0140]. Online: tuned rho / cF 6.1232 [4.1522, 11.0853] (cB: 6.4521 [4.3774, 11.7019]). Distance remaining, not a gate.
- **Rank stage (secondary; two blocks of 600 attempts per arm).** Per-attempt cost kB / kN 2.2451 [2.0517, 2.4650] (mean attempt 1,262.0 ms against 562.1 ms; probes per attempt 2,480,245 against 1,678,903); whole stage kB / kN 2.2447. Cold S with the rank stage added: old (kB + cB) 30.855; fold only (kB + cF) 30.117; fold and rank (kN + cF) 13.906. The cold gap to tuned rho including the rank stage: 243.3 (old), 237.4 (fold only), 109.6 (fold and rank). The rank stage still dominates cold cost with the fold alone; it is the next stage to cut.

Limits of this result

- Tier 1, instance scope: one runner, one pinned CPU, 16 points, one fresh process per arm per point. The intervals are over those 16 points.
- The baseline cB is the combined producer (lift_scan x Feistel). That combination's own frozen primary was `not_separated` against its better single arm (PR #929); the single-arm pairs above show the fold gives the same cold gain on each parent.
- The cold figure here excludes the rank stage (it is measured separately and reported above), and its cold gap to rT is much smaller than a cold gap that includes the rank stage.

Files (results)

File	Role
`run_github_38819494806/`	the archived artifact: allocation, builds, raw runs, `panel/result.json`
`make_figures.py`, `figures/`	result figure (TikZ source, PDF, PNG)
`README.pdf`	this report: `pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true`